Basi di dati

Esercitazione sulla ristrutturazione dello schema logico

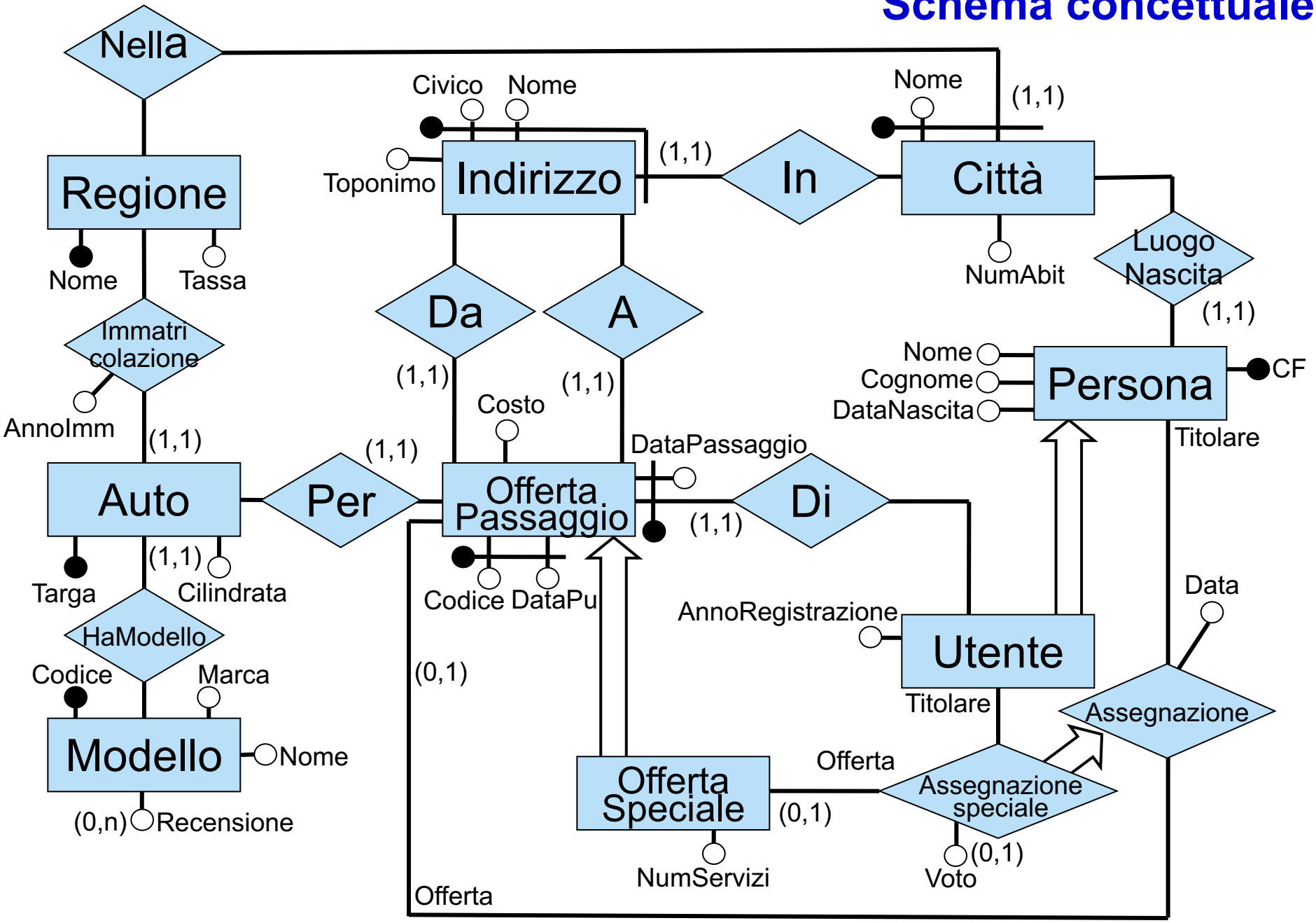
Anno Accademico 2018/2019

Maurizio Lenzerini

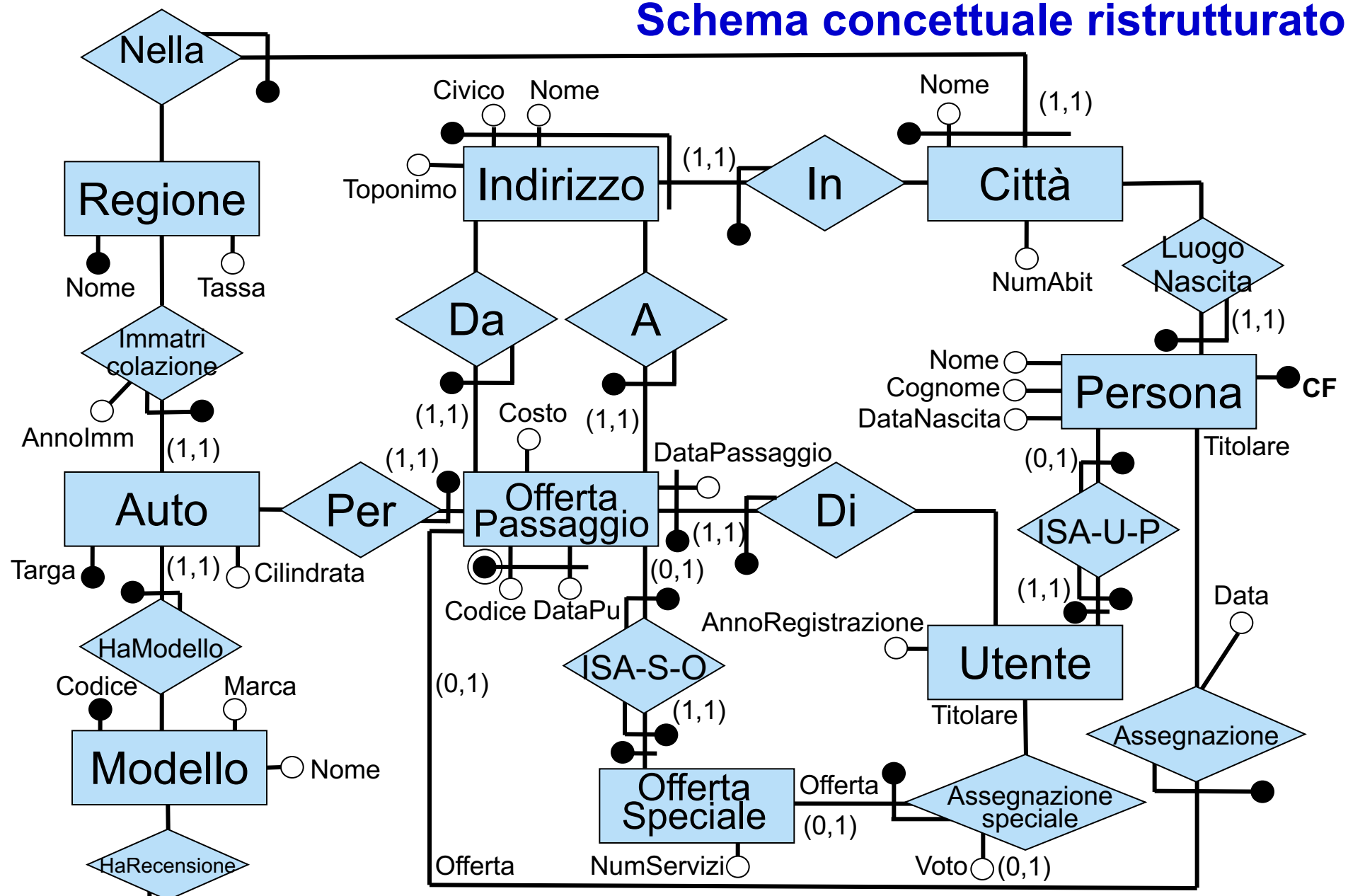
Specifica dei requisiti

Si richiede di effettuare la progettazione concettuale relativa al sistema informativo di un'azienda di "car pooling". Gli utenti registrati al servizio di "car pooling" pubblicano su Web offerte di passaggi con automobili private, dove ogni offerta è caratterizzata da: l'utente registrato (una persona) che l'ha pubblicata, la data di pubblicazione, un codice numerico (unico nell'ambito della data di pubblicazione), la data in cui avverrà il passaggio, il costo del passaggio, l'automobile che si utilizzerà per il passaggio, l'indirizzo da cui l'automobile partirà e l'indirizzo a cui l'automobile arriverà. Si noti che un utente registrato non può offrire più di un passaggio per lo stesso giorno. Alla prima persona che aderisce ad un'offerta viene assegnato il servizio: tale persona diventa il titolare dell'assegnazione del servizio ed ha quindi diritto al passaggio. Di ogni assegnazione interessa l'offerta alla quale si riferisce, la data in cui è avvenuta e la persona titolare dell'assegnazione. Di ogni automobile interessa la targa (identificativo), la cilindrata, l'anno di immatricolazione, la regione di immatricolazione, ed il modello. Di ogni modello interessa il codice (identificativo), il nome (ad esempio, "Fiat 500"), la marca (ad esempio "Fiat") e le recensioni ricevute. Di ogni persona interessa il codice fiscale (identificativo), il nome, il cognome, la data di nascita e la città di nascita. Di ogni utente registrato interessa il codice fiscale (identificativo), il nome, il cognome, la data di nascita, la città di nascita e l'anno di registrazione al servizio. Ogni indirizzo è caratterizzato dal toponimo (via, piazza, viale, ecc.), dal nome, dal numero civico e dalla città. In nessuna città si trovano due indirizzi con lo stesso nome e lo stesso numero civico. Di ogni città interessa il nome (unico nell'ambito della regione), la regione ed il numero di abitanti, e di ogni regione interessa il nome (identificativo) e la tassa che la regione applica per ogni passaggio. Alcune offerte sono speciali, nel senso che offrono servizi in più (ad esempio, l'wifi), e delle offerte speciali interessa sapere quanti sono i servizi in più che esse offrono. Si noti che solo gli utenti registrati possono aderire alle offerte speciali. Quindi i titolari di servizi relativi ad offerte speciali sono necessariamente utenti registrati, e questi, dopo il passaggio, possono attribuire un voto al servizio, voto che è di interesse all'applicazione.

Schema concettuale

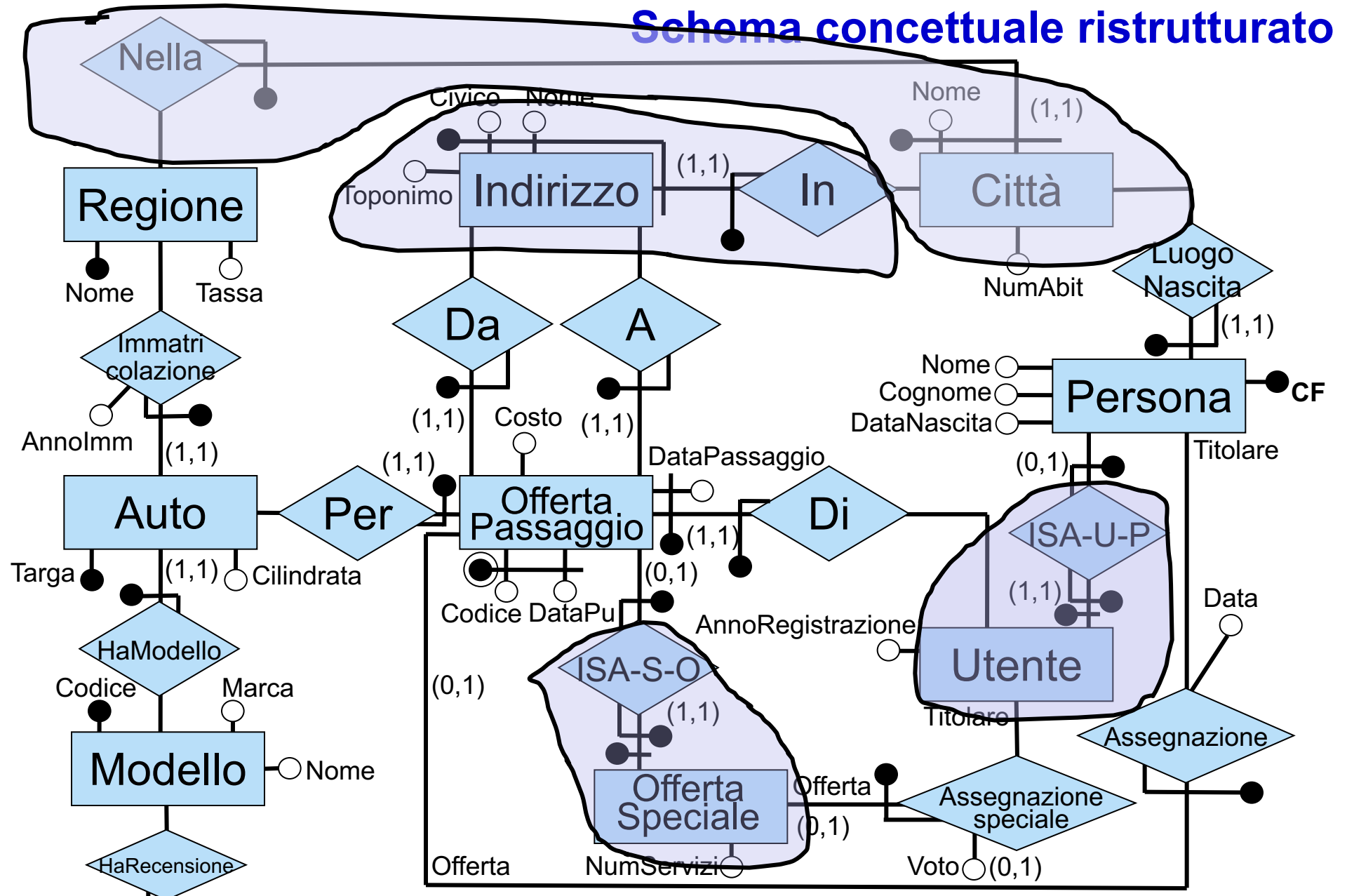


Schema concettuale ristrutturato



Vincolo esterno: Per ogni istanza <Offerta:o,Titolare:t> di AssegnazioneSpeciale, se o1 e t1 sono tali che <OffertaSpeciale:o,OffertaPassaggio:o1> è istanza di ISA-S-O e <Utente:t,Persona:t1> è istanza di ISA-U-P, si ha che l'stanza <Offerta:o1,Titolare:t1> è istanza di Assegnazione.

Schema concettuale ristrutturato



Vincolo esterno: Per ogni istanza <Offerta:o,Titolare:t> di AssegnazioneSpeciale, se o1 e t1 sono tali che <OffertaSpeciale:o,OffertaPassaggio:o1> è istanza di ISA-S-O e <Utente:t,Persona:t1> è istanza di ISA-U-P, si ha che l'stanza <Offerta:o1,Titolare:t1> è istanza di Assegnazione.

Traduzione diretta (1)

Recensione(<u>testo</u>)
  foreign key: Recensione[testo] $\subseteq$ HaRecensione[rec]
HaRecensione(<u>rec</u>,modello)
  foreign key: HaRecensione[rec] $\subseteq$ Recensione[testo]
  foreign key: HaRecensione[modello] $\subseteq$ Modello[codice]
Modello(<u>codice</u>,marca,nome)
HaModello(<u>auto</u>,modello)
  foreign key: HaModello[auto] $\subseteq$ Auto[targa]
  foreign key: HaModello[modello] $\subseteq$ Modello[codice]
Auto(<u>targa</u>,cilindrata)
  foreign key: Auto[targa] $\subseteq$ Immatricolazione[auto]
  foreign key: Auto[targa] $\subseteq$ HaModello[auto]
Immatricolazione(<u>auto</u>,regione,annoImm)
  foreign key: Immatricolazione[auto] $\subseteq$ Auto[targa]
  foreign key: Immatricolazione[regione] $\subseteq$ Regione[nome]
Regione(<u>nome</u>,tassa)
Città(<u>nome</u>,<u>regione</u>,numAbit)
  foreign key: Città[regione] $\subseteq$ Regione[nome]
Indirizzo(<u>nome</u>,<u>civico</u>,<u>nomeCittà</u>,<u>regione</u>,toponimo)
  foreign key: Indirizzo[nomeCittà,regione] $\subseteq$ Città[nome,regione]
Persona(<u>CF</u>,nome,cognome,dataNascita)
  foreign key: Persona[CF] $\subseteq$ LuogoNascita[persona]
LuogoNascita(<u>persona</u>,nomeCittà,regione)
  foreign key: LuogoNascita[persona] $\subseteq$ Persona[CF]
  foreign key: LuogoNascita[nomeCittà,regione] $\subseteq$ Città[nome,regione]
Utente(<u>CF</u>,annoRegistrazione)
  foreign key: Utente[CF] $\subseteq$ Persona[CF]

Traduzione diretta (2)

OffertaPassaggio(codice,dataPub,costo,dataPassaggio)
- foreign key: OffertaPassaggio[codice,dataPub] ⊆ Per[codice,dataPub]
- foreign key: OffertaPassaggio[codice,dataPub] ⊆ Da[codice,dataPub]
- foreign key: OffertaPassaggio[codice,dataPub] ⊆ A[codice,dataPub]
- foreign key: OffertaPassaggio[codice,dataPub] ⊆ Di[codice,dataPub]

Per(codice,dataPub,auto)
- foreign key: Per[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
- foreign key: Per[auto] ⊆ Auto[targa]

Da(codice,dataPub,nome,civico,nomeCittà,regione)
- foreign key: Da[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
- foreign key: Da[nome,civico,nomeCittà,regione] ⊆ Indirizzo[nome,civico,nomeCittà,regione]

A(codice,dataPub,nome,civico,nomeCittà,regione)
- foreign key: A[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
- foreign key: A[nome,civico,nomeCittà,regione] ⊆ Indirizzo[nome,civico,nomeCittà,regione]

Di(codice,dataPub,utente)
- foreign key: Di[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
- foreign key: Di[utente] ⊆ Utente[CF]

OffertaSpeciale(codice,dataPub,numServizi)
- foreign key: OffertaSpeciale[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]

Assegnazione(codOfferta,dataPubOff,titolare,data)
- foreign key: Assegnazione[codOfferta,dataPubOff] ⊆ OffertaPassaggio[codice,dataPub]
- foreign key: Assegnazione[titolare] ⊆ Persona[CF]

AssegnazioneSpeciale(codOfferta,dataPubOff,titolare,voto*)
- foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff,titolare] ⊆ Assegnazione[codOfferta,dataPubOff,titolare]
- foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff] ⊆ OffertaSpeciale[codOfferta,dataPubOff]
- foreign key: AssegnazioneSpeciale[titolare] ⊆ Utente[CF]

Vincolo esterno: nel join naturale tra Di e OffertaPassaggio non esistono due tuple con la stessa combinazione di valori per dataPassaggio e utente

Nota: il vincolo esterno è diventato interno, perché corrisponde alla foreign key tra AssegnazioneSpeciale e Assegnazione

Scema logico da traduzione diretta

Recensione(testo)
foreign key: Recensione[testo] ⊆ HaRecensione[rec]
HaRecensione(rec,modello)
foreign key: HaRecensione[rec] ⊆ Recensione[testo]
foreign key: HaRecensione[modello] ⊆ Modello[codice]
Modello(codice,marca,nome)
HaModello(auto,modello)
foreign key: HaModello[auto] ⊆ Auto[targa]
foreign key: HaModello[modello] ⊆ Modello[codice]
Auto(targa,cilindrata)
foreign key: Auto[targa] ⊆ Immatricolazione[auto]
foreign key: Auto[targa] ⊆ HaModello[auto]
Immatricolazione(auto,regione,annoImm)
foreign key: Immatricolazione[auto] ⊆ Auto[targa]
foreign key: Immatricolazione[regione] ⊆ Regione[nome]
Regione(nome,tassa)
Città(nome,regione,numAbit)
foreign key: Città[regione] ⊆ Regione[nome]
Indirizzo(nome,civico,nomeCittà,regione,toponimo)
foreign key: Indirizzo[nomeCittà,regione] ⊆ Città[nome,regione]
Persona(CF,nome,cognome,dataNascita)
foreign key: Persona[CF] ⊆ LuogoNascita[persona]
LuogoNascita(persona,nomeCittà,regione)
foreign key: LuogoNascita[persona] ⊆ Persona[CF]
foreign key: LuogoNascita[nomeCittà,regione] ⊆ Città[nome,regione]
Utente(CF,annoRegistrazione)
foreign key: Utente[CF] ⊆ Persona[CF]
OffertaPassaggio(codice,dataPub,costo,dataPassaggio)
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Per[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Da[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ A[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Di[codice,dataPub]

Per(codice,dataPub,auto)
foreign key: Per[auto] ⊆ Auto[targa]
foreign key: Per[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
Da(codice,dataPub,nome,civico,nomeCittà,regione)
foreign key: Da[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Da[nome,civico,nomeCittà,regione] ⊆ Indirizzo[nome,civico,nomeCittà,regione]
A(codice,dataPub,nome,civico,nomeCittà,regione)
foreign key: A[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: A[nome,civico,nomeCittà,regione] ⊆ Indirizzo[nome,civico,nomeCittà,regione]
Di(codice,dataPub,utente)
foreign key: Di[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Di[utente] ⊆ Utente[CF]
OffertaSpeciale(codice,dataPub,numServizi)
foreign key: OffertaSpeciale[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
Assegnazione(codOfferta,dataPubOff,titolare,data)
foreign key: Assegnazione[codOfferta,dataPubOff] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Assegnazione[titolare] ⊆ Persona[CF]
AssegnazioneSpeciale(codOfferta,dataPubOff,titolare,voto*)
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff,titolare] ⊆ Assegnazione[codOfferta,dataPubOff,titolare]
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff] ⊆ OffertaSpeciale[codOfferta,dataPubOff]
foreign key: AssegnazioneSpeciale[titolare] ⊆ Utente[CF]

Vincolo esterno: nel join naturale tra Di e OffertaPassaggio non esistono due tuple con la stessa combinazione di valori per dataPassaggio e utente

Ristrutturazione dello schema logico: indicazioni di progetto

Si ristrutturi lo schema logico tenendo conto delle seguenti indicazioni:

- Quando si accede ad un'auto si vuole sempre conoscere la regione, l'anno di immatricolazione ed il modello
- Quando si accede ad una persona si vuole sempre conoscere il luogo di nascita, se è un utente e, se lo è, l'anno di registrazione; quando si accede ad un utente si vogliono sempre conoscere, oltre all'anno di registrazione, i dati anagrafici (nome, cognome, data di nascita, luogo di nascita)
- Quando si accede ai dati relativi ad una offerta si vuole sempre conoscere la relativa auto e l'utente che ha pubblicato l'offerta
- Quando si vuole conoscere l'indirizzo di partenza di una offerta si vuole anche sempre conoscere l'indirizzo di arrivo, e viceversa, quando si vuole conoscere l'indirizzo di arrivo di una offerta si vuole anche sempre conoscere l'indirizzo di partenza

Ristrutturazione dello schema logico

Le ristrutturazioni dello schema logico che dobbiamo prevedere sono:

- Quando si accede ai dati relativi ad un'auto si vuole sempre conoscere la regione, l'anno di immatricolazione ed il modello

  → Accorpamento: si accorpano le tabelle Auto e Immatricolazione, che sono fortemente accoppiate, e le tabelle Auto e HaModello, anch'esse fortemente accoppiate

- Quando si accede ad una persona si vuole spesso conoscere il luogo di nascita della persona, se è un utente e, se lo è, l'anno di registrazione; quando si accede ai dati relativi ad un utente si vogliono sempre conoscere, oltre all'anno di registrazione, i dati anagrafici (nome, cognome, data di nascita, luogo di nascita) dell'utente

  → Accorpamento: si accorpano le tabelle Persona e LuogoNascita, fortemente accoppiate, in modo e le tabelle Persona e Utente, debolmente accoppiate).

- Quando si accede ai dati relativi ad una offerta si vuole sempre la relativa auto e l'utente che ha pubblicato l'offerta

  → Accorpamento: si accorpa la tabella Per alla tabella OffertaPassaggio, fortemente accoppiate, e la tabella Di alla tabella OffertaPassaggio, anch'esse fortemente accoppiate.

- Quando si vuole conoscere l'indirizzo di partenza di una offerta si vuole anche sempre conoscere l'indirizzo di arrivo, e viceversa, quando si vuole conoscere l'indirizzo di arrivo di una offerta si vuole anche sempre conoscere l'indirizzo di partenza

  → Accorpamento: si accorpano le tabelle Da e A, fortemente accoppiate (sono fortemente accoppiate perché, anche se le due foreign key reciprochce non sono esplicite, esse valgono, visto che entrambe le tabelle sono fortemente accoppiate con la tabella OffertaPassaggio)

- Eliminazione della tabella inutile Recensione

Schema logico ristrutturato

HaRecensione(rec,modello)
foreign key: HaRecensione[modello] ⊆ Modello[codice
Modello(codice,marca,nome)
Auto(targa,cilindrata,regione,annoImm,modello)
foreign key: Auto[modello] ⊆ Modello[codice]
foreign key: Auto[regione] ⊆ Regione[nome]
Regione(nome,tassa)
Città(nome,regione,numAbit)
foreign key: Città[regione] ⊆ Regione[nome]
Indirizzo(nome,civico,nomeCittà,regione,toponimo)
foreign key: Indirizzo[nomeCittà,regione] ⊆ Città[nome,regione]
Persona(CF,nome,cognome,dataNascita,nomeCitNascita,regioneNascita,annoRegistrazione*)
foreign key: Persona[nomeCItNascita,regioneNascita] ⊆ Città[nome,regione]
OffertaPassaggio(codice,dataPub,costo,dataPassaggio,auto,utente)
foreign key: OffertaPassaggio[auto] ⊆ Auto[targa]
inclusione: Offerta[utente] ⊆ (select CF from Persona where annoRegistrazione is not null)
chiave: utente, dataPassaggio
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Tragitto[codice,dataPub]
Tragitto(codice,dataPub,nomeDa,civicoDa,nomeCittàDa,regioneDa,nomeA,civicoA,nomeCittàA,regioneA)
foreign key: Da[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Da[nomeDa,civicoDa,nomeCittàDa,regioneDa] ⊆ Indirizzo[nome,civico,nomeCittà,regione]
foreign key: Da[nomeA,civicoA,nomeCittàA,regioneA] ⊆ Indirizzo[nome,civico,nomeCittà,regione]
OffertaSpeciale(codice,dataPub,numServizi)
foreign key: OffertaSpeciale[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
Assegnazione(codOfferta,dataPubOff,titolare,data)
foreign key: Assegnazione[codOfferta,dataPubOff] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Assegnazione[titolare] ⊆ Persona[CF]
AssegnazioneSpeciale(codOfferta,dataPubOff,titolare,voto*)
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff,titolare] ⊆ Assegnazione[codOfferta,dataPubOff,titolare]
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff] ⊆ OffertaSpeciale[codOfferta,dataPubOff]
fnclusioe: AssegnazioneSpeciale[titolare] ⊆ (select CF from Persona where annoRegistrazione is not null)

Schema logico da traduzione diretta

Recensione(testo)
foreign key: Recensione[testo] ⊆ HaRecensione[rec]
HaRecensione(rec,modello)
foreign key: HaRecensione[rec] ⊆ Recensione[testo]
foreign key: HaRecensione[modello] ⊆ Modello[codice]
Modello(codice,marca,nome)
HaModello(auto,modello)
foreign key: HaModello[auto] ⊆ Auto[targa]
foreign key: HaModello[modello] ⊆ Modello[codice]
Auto(targa,cilindrata)
foreign key: Auto[targa] ⊆ Immatricolazione[auto]
foreign key: Auto[targa] ⊆ HaModello[auto]
Immatricolazione(auto,regione,annoImm)
foreign key: Immatricolazione[auto] ⊆ Auto[targa]
foreign key: Immatricolazione[regione] ⊆ Regione[nome]
Regione(nome,tassa)
Città(nome,regione,numAbit) foreign key: Città[regione] ⊆ Regione[nome]
Indirizzo(nome,civico,nomeCittà,regione,toponimo)
foreign key: Indirizzo[nomeCittà,regione] ⊆ Città[nome,regione]
Persona(CF,nome,cognome,dataNascita)
foreign key: Persona[CF] ⊆ LuogoNascita[persona]
LuogoNascita(persona,nomeCittà,regione)
foreign key: LuogoNascita[persona] ⊆ Persona[CF]
foreign key: LuogoNascita[nomeCittà,regione] ⊆ Città[nome,regione]
Utente(CF,annoRegistrazione)
foreign key: Utente[CF] ⊆ Persona[CF]
OffertaPassaggio(codice,dataPub,costo,dataPassaggio)
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Per[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Da[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ A[codice,dataPub]
foreign key: OffertaPassaggio[codice,dataPub] ⊆ Di[codice,dataPub]
Per(codice,dataPub,auto) foreign key: Per[auto] ⊆ Auto[targa]
foreign key: Per[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
Da(codice,dataPub,nome,civico,nomeCittà,regione)
foreign key: Da[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Da[nome,civico,nomeCittà,regione] ⊆
Indirizzo[nome,civico,nomeCittà,regione]
A(codice,dataPub,nome,civico,nomeCittà,regione)
foreign key: A[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: A[nome,civico,nomeCittà,regione] ⊆
Indirizzo[nome,civico,nomeCittà,regione]
Di(codice,dataPub,utente)
foreign key: Di[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Di[utente] ⊆ Utente[CF]
OffertaSpeciale(codice,dataPub,numServizi)
foreign key: OffertaSpeciale[codice,dataPub] ⊆
OffertaPassaggio[codice,dataPub]
Assegnazione(codOfferta,dataPubOff,titolare,data)
foreign key: Assegnazione[codOfferta,dataPubOff] ⊆
OffertaPassaggio[codice,dataPub]
foreign key: Assegnazione[titolare] ⊆ Persona[CF]
AssegnazioneSpeciale(codOfferta,dataPubOff,titolare,voto*)
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff,titolare] ⊆
Assegnazione[codOfferta,dataPubOff,titolare]
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff] ⊆
OffertaSpeciale[codOfferta,dataPubOff]
foreign key: AssegnazioneSpeciale[titolare] ⊆ Utente[CF]

Vincolo esterno: nel join naturale tra Di e OffertaPassaggio non esistono due tuple con la stessa combinazione di valori per dataPassaggio e utente

Schema logico ristrutturato

HaRecensione(rec,modello)
foreign key: HaRecensione[modello] ⊆ Modello[codice]
Modello(codice,marca,nome)
Auto(targa,cilindrata,regione,annoImm,modello)
foreign key: Auto[modello] ⊆ Modello[codice]
foreign key: Auto[regione] ⊆ Regione[nome]
Regione(nome,tassa)
Città(nome,regione,numAbit)
foreign key: Città[regione] ⊆ Regione[nome]
Indirizzo(nome,civico,nomeCittà,regione,toponimo)
foreign key: Indirizzo[nomeCittà,regione] ⊆ Città[nome,regione]
Persona(CF,nome,cognome,dataNascita,nomeCitNascita,regioneNascita,annoRegistrazione*)
foreign key: Persona[nomeCItNascita,regioneNascita] ⊆
Città[nome,regione]
OffertaPassaggio(codice,dataPub,costo,dataPassaggio,auto,utente)
foreign key: OffertaPassaggio[auto] ⊆ Auto[targa]
vincolo: Offerta[utente] ⊆
(select CF from Persona where annoRegistrazione is not null)
chiave: utente, dataPassaggio
foreign key: OffertaPassaggio[codice,dataPub] ⊆
Tragitto[codice,dataPub]
Tragitto(codice,dataPub,nomeDa,civicoDa,nomeCittàDa,regioneDa,nomeA,civicoA,nomeCittàA,regioneA)
foreign key: Da[codice,dataPub] ⊆ OffertaPassaggio[codice,dataPub]
foreign key: Da[nomeDa,civicoDa,nomeCittàDa,regioneDa] ⊆
Indirizzo[nome,civico,nomeCittà,regione]
foreign key: Da[nomeA,civicoA,nomeCittàA,regioneA] ⊆
Indirizzo[nome,civico,nomeCittà,regione]
OffertaSpeciale(codice,dataPub,numServizi)
foreign key: OffertaSpeciale[codice,dataPub] ⊆
OffertaPassaggio[codice,dataPub]
Assegnazione(codOfferta,dataPubOff,titolare,data)
foreign key: Assegnazione[codOfferta,dataPubOff] ⊆
OffertaPassaggio[codice,dataPub]
foreign key: Assegnazione[titolare] ⊆ Persona[CF]
AssegnazioneSpeciale(codOfferta,dataPubOff,titolare,voto*)
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff,titolare] ⊆
Assegnazione[codOfferta,dataPubOff,titolare]
foreign key: AssegnazioneSpeciale[codOfferta,dataPubOff] ⊆
OffertaSpeciale[codOfferta,dataPubOff]
foreign key: AssegnazioneSpeciale[titolare] ⊆
(select CF from Persona where annoRegistrazione is not null)